

Mega Arena Royale

Guia do Jogador e Guia do Desenvolvedor

Tiago Linhares Medeiros

Mega Arena Royale

Mega Arena Royale é um jogo de tiro 2D em arena, feito em um único arquivo HTML: `jogodetiro.html`.

Autoria: Tiago Linhares Medeiros.

O jogo mistura elementos de battle royale, progressão permanente, loja, skins, passe de batalha e um modo contra chefe. Ele roda diretamente no navegador, sem servidor, sem build e sem dependências externas.

Para Jogadores

Objetivo

Sobreviva na arena, elimine inimigos, colete melhorias e vença a partida conforme o modo escolhido.

O jogo tem três modos principais:

- **Solo:** você enfrenta 9 bots. Vence se for o último sobrevivente.
- **Duplas:** você joga em equipe. Mesmo se morrer, pode assistir ao aliado restante.
- **Chefe:** você e aliados enfrentam um Titã. O objetivo é derrotar o chefe antes que sua equipe seja eliminada.

Controles

Ação	Controle
Mover	W, A, S, D
Mirar	Mouse
Atirar	Clique esquerdo
Espada	1
Granada	2
Basuca	3
Esquiva	Espaço

Regras Básicas

- Cada personagem tem HP, escudo, dano e velocidade.
- Tiros, espada, granadas e basuca causam dano aos inimigos.
- Acertos mais precisos podem causar dano crítico.
- Se o escudo estiver ativo, ele absorve dano antes do HP.
- A partida termina quando você, sua equipe ou os inimigos cumprem a condição de vitória/derrota do modo.

Zona Segura

Nos modos **Solo** e **Duplas**, existe uma zona segura circular.

- A zona diminui com o tempo.
- Ficar fora dela causa dano periódico.
- Bots também sofrem dano fora da zona segura.

- No modo **Chefão**, a zona segura fica desativada na prática para priorizar a batalha contra o Titã.

Caixas da Arena

Durante a partida, caixas aparecem em pontos fixos da arena. Ao coletar uma caixa, você recebe um bônus:

Tipo	Efeito
Vida	Recupera HP
Escudo	Adiciona escudo
Arma	Aumenta o dano
Velocidade	Aumenta a velocidade

Bots também podem coletar caixas.

Loja e Progressão

Ao final das partidas, você ganha moedas conforme seu desempenho.

Na loja, moedas podem ser usadas para comprar:

- Rifle Dourado: aumenta o dano inicial.
- Colete Tático: adiciona escudo inicial.
- Ninja: aumenta velocidade.
- Espada de Plasma: libera ataque corpo a corpo.
- Granadas: libera explosivos.
- Módulo de Regeneração: recupera vida aos poucos.
- Botas Táticas: aumenta velocidade.
- Coração de Titã: aumenta HP máximo.
- Vampirismo: recupera vida ao eliminar inimigos.
- Skins: mudam a aparência do personagem.

Passe de Batalha

O jogo possui missões e recompensas por patamar.

Missões atuais:

- Jogar partidas.
- Eliminar inimigos.
- Causar dano.
- Vencer partidas ou derrotar o chefe.

Ao completar missões, você ganha diamantes. Diamantes liberam patamares do passe. O passe VIP melhora recompensas e pode liberar uma skin especial.

Recompensas de Fim de Partida

As moedas ganhas dependem de:

- modo de jogo;
- colocação final;
- vitória ou derrota;
- dificuldade escolhida;
- progresso no modo chefe.

Quanto maior a dificuldade, maior o multiplicador de moedas.

Para Desenvolvedores

Como Executar

Abra o arquivo diretamente no navegador:

`<pasta-do-jogo>/jogodetiro.html`

Substitua `<pasta-do-jogo>` pela pasta onde o arquivo foi salvo. Evite registrar caminhos pessoais do computador na documentação final.

Não há instalação, servidor local, empacotador ou dependências externas.

Estrutura Geral

O projeto está concentrado em um único arquivo:

`jogodetiro.html`

Esse arquivo contém:

- HTML dos menus, HUD, loja, passe e tela de fim de jogo.
- CSS da interface visual.
- JavaScript com estado do jogo, input, IA dos bots, física simples, combate, renderização e persistência.

Principais Sistemas

Sistema	Descrição
Save	Usa <code>localStorage</code> para moedas, diamantes, inventário, passe e progresso do chefão.
Menu	Controla tela principal, loja, passe e fim de jogo.
Input	Usa <code>WASD</code> , mouse, clique e teclas numéricas.
Arena	Usa canvas 2D com dimensão de 1500 x 1500.
Paredes	Definidas no array <code>walls</code> .
Bots	Criados em <code>iniciarJogo()</code> conforme modo e dificuldade.
Combate	Usa arrays como <code>bullets</code> , <code>bombs</code> , <code>basucas</code> , <code>slashes</code> e <code>explosions</code> .
UI	Atualizada por <code>atualizarUI()</code> e pelos menus.
Passe	Gerado dinamicamente por <code>gerarPasseHTML()</code> .
Renderização	Feita em <code>draw()</code> a cada frame.
Loop principal	<code>gameLoop()</code> chama <code>update()</code> e <code>draw()</code> .

Estado Persistente

O jogo salva progresso no navegador com estas chaves:

```
br_moedas  
br_diamantes  
br_inventario  
br_nivel_chefe  
br_passe
```

Para resetar o progresso durante testes, limpe essas chaves no `localStorage` do navegador.

Exemplo no console:

```
localStorage.removeItem("br_moedas");  
localStorage.removeItem("br_diamantes");  
localStorage.removeItem("br_inventario");  
localStorage.removeItem("br_nivel_chefe");
```

```
localStorage.removeItem("br_passe");  
location.reload();
```

Fluxo de Uma Partida

1. O jogador escolhe modo e dificuldade.
2. `iniciarJogo()` cria jogador, bots, caixas, arena e estado inicial.
3. O jogo entra em `COUNTDOWN`.
4. Depois da contagem, o estado muda para `PLAYING`.
5. `update()` processa movimento, tiros, IA, dano, zona segura, caixas e condições de fim.
6. `draw()` redesenha arena, personagens, projéteis, efeitos e HUD visual.
7. Quando a partida termina, `finalizarPartida()` calcula recompensas, atualiza missões e salva progresso.

Modos de Jogo no Código

O modo atual fica na variável:

`modoJogo`

Valores esperados:

`solo`
`duo`
`boss`

No modo `boss`, o jogo cria aliados e um chefe com HP maior. Nos modos `solo` e `duo`, o jogo cria bots comuns e ativa a zona segura.

Pontos Importantes para Atualizar

Ao alterar o jogo, tenha cuidado com:

- `inventario`: afeta jogador, loja, habilidades e skins.
- `passe`: controla missões, patamares e recompensas.
- `bots`: bots e aliados usam a mesma lista.
- `teamId`: usado para impedir dano entre aliados.
- `safeZone.radius`: controla a zona segura.
- `player.angle`: agora depende da mira do mouse.
- `localStorage`: mudanças de estrutura podem quebrar saves antigos.

Ideias de Evolução

Possíveis melhorias futuras:

- Separar HTML, CSS e JavaScript em arquivos distintos.
- Criar um sistema de armas com configurações em objeto.
- Melhorar IA dos bots com busca A*, comportamento tático e cobertura.
- Criar mapas diferentes.
- Adicionar tela de pausa.
- Adicionar tutorial inicial.
- Adicionar balanceamento por dificuldade.
- Criar testes simples para funções de dano, compra e progresso.

Ideias de IA para Trabalhos Práticos

Este jogo pode ser usado como base para atividades de inteligência artificial:

- Busca informada: bots usando A* para desviar de paredes e chegar a caixas, inimigos ou zona segura.
- Busca online/local: bots que recalculam decisões em tempo real com base em informação parcial.
- Busca competitiva: bots ou chefe usando Minimax, Alpha-Beta ou funções heurísticas para escolher entre atacar, fugir, pegar item ou buscar cobertura.

Checklist Antes de Entregar Mudanças

Use este checklist após modificar o código:

- O menu principal abre sem erros.
- O modo Solo inicia corretamente.
- WASD, mouse e clique funcionam.
- Loja abre e botões não quebram.
- Passe de batalha abre e lista missões/recompensas.
- Uma partida termina sem travar.
- O progresso continua salvando no `localStorage`.
- O JavaScript passa em uma checagem de sintaxe.

Com Node.js instalado, a checagem de sintaxe pode ser feita extraindo o script do HTML:

```
perl -One 'print $1 if /<script>(.*?)<\/script>/s' jogodetiro.html | node --check
```

Observações

Este projeto prioriza simplicidade e facilidade de demonstração. Por isso, tudo ainda está em um arquivo HTML único. Para evolução maior, o primeiro passo recomendado é separar responsabilidades sem alterar a jogabilidade:

- `index.html` para estrutura;
- `styles.css` para visual;
- `game.js` para lógica principal;
- `ai.js` para comportamento dos bots;
- `save.js` para persistência.